

BANCO DE DADOS

Trabalho – Relatório

Curso:	Análise e Desenvolvimento de Sistemas
Aluno(a):	Esther Mara Balduino de Paulo
RU:	

1. 1ª Etapa – Modelagem

Pontuação: 25 pontos.

Dadas as regras de negócio abaixo listadas, referentes ao estudo de caso de uma companhia aérea, elabore o Modelo Entidade-Relacionamento (MER), isto é, o modelo conceitual.

O Modelo Entidade-Relacionamento (MER) deve contemplar os seguintes itens:

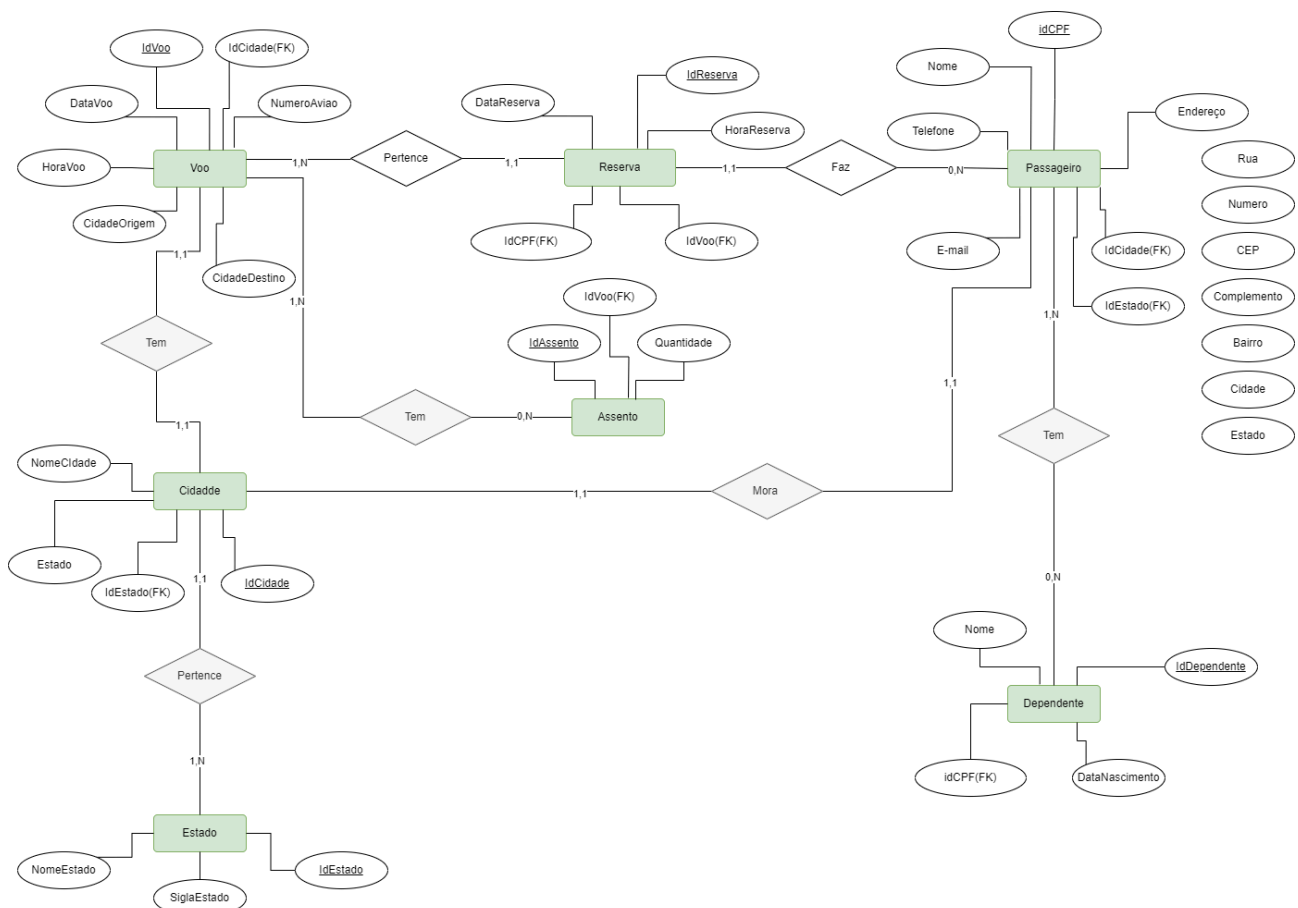
- Entidades;
- Atributos;
- Relacionamentos;
- Cardinalidades;
- Chaves primárias;
- Chaves estrangeiras.

Uma companhia aérea necessita controlar os dados de seus voos. Para isso, contratou um profissional de Banco de Dados, a fim de modelar o Banco de Dados que armazenará os dados dos voos.

As regras de negócio são:

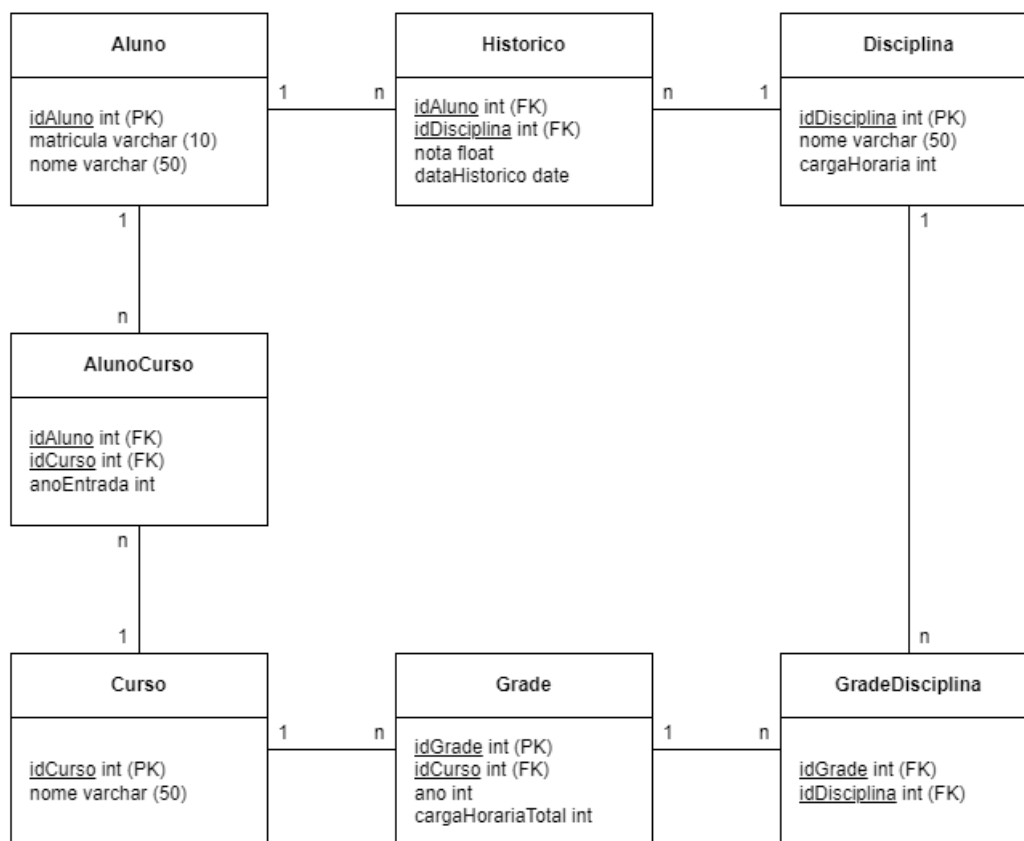
- Voo – Deverão ser armazenados os seguintes dados: identificação do voo, número do avião, cidade de origem, cidade de destino, data do voo e hora do voo;

- Assento – Deverão ser armazenados os seguintes dados: identificação do assento e quantidade;
- Passageiro – Deverão ser armazenados os seguintes dados: CPF, nome, telefone, e-mail e endereço (rua, número, complemento, bairro, CEP, cidade e estado);
- Dependente – Deverão ser armazenados os seguintes dados: nome e data de nascimento;
- Um voo pode ter zero ou vários assentos, assim como zero ou vários assentos pertencem a um voo;
- Um passageiro pode ter zero ou várias reservas de assentos, assim como zero ou várias reservas de assentos pertencem a um passageiro;
- Um passageiro pode ter zero ou vários dependentes, assim como zero ou vários dependentes são de um passageiro;
- Da reserva deverão ser armazenados os seguintes dados: data da reserva e hora da reserva.



2. 2ª Etapa – Implementação

Considere o seguinte Modelo Relacional (modelo lógico), referente ao estudo de caso de uma faculdade:



Com base no Modelo Relacional dado e utilizando a *Structured Query Language* (SQL), no MySQL Workbench, implemente o que se pede.

Observação: Para testar o Banco de Dados após a implementação, utilize os comandos contidos no arquivo “Trabalho – Populando o Banco de Dados” para popular as

tabelas. Tal arquivo contém todos os comandos de inserção dos dados (fictícios) necessários para a realização dos testes.

Pontuação: 25 pontos.

1. Implemente um Banco de Dados chamado “Faculdade”. Após, implemente as tabelas, conforme o Modelo Relacional dado, observando as chaves primárias e as chaves estrangeiras. Todos os campos, de todas as tabelas, não podem ser nulos (*not null*).

CREATE DATABASE Faculdade;

USE Faculdade;

**CREATE TABLE Aluno (
 idAluno int unsigned NOT NULL auto_increment,
 matricula varchar(10) NOT NULL,
 nome varchar(50) NOT NULL,
 PRIMARY KEY (idAluno)
);**

**CREATE TABLE Curso (
 idCurso int unsigned NOT NULL auto_increment,
 nome varchar(50) NOT NULL,
 PRIMARY KEY (idCurso)
);**

**CREATE TABLE Disciplina (
 idDisciplina int unsigned NOT NULL auto_increment,
 nome varchar(50) NOT NULL,
 cargaHoraria int NOT NULL,**

PRIMARY KEY (idDisciplina)

);

CREATE TABLE AlunoCurso (

id int unsigned NOT NULL auto_increment,

idAluno int unsigned NOT NULL,

idCurso int unsigned NOT NULL,

anoEntrada int NOT NULL,

PRIMARY KEY (id),

CONSTRAINT fk_aluno FOREIGN KEY (idAluno) REFERENCES aluno(idAluno),

CONSTRAINT fk_curso FOREIGN KEY (idCurso) REFERENCES curso(idCurso)

);

CREATE TABLE Grade (

idGrade int unsigned NOT NULL auto_increment,

idCurso int unsigned NOT NULL,

ano int NOT NULL,

cargaHorariaTotal int DEFAULT NULL,

PRIMARY KEY (idGrade),

constraint fk_grade_curso foreign key (idCurso) references curso(idCurso)

);

CREATE TABLE GradeDisciplina (

id int unsigned NOT NULL auto_increment,

idGrade int unsigned NOT NULL,

idDisciplina int unsigned NOT NULL,

PRIMARY KEY (id),

constraint fk_grade foreign key (idGrade) references grade(idGrade),

constraint fk_disciplina foreign key (idDisciplina) references disciplina(idDisciplina)

);

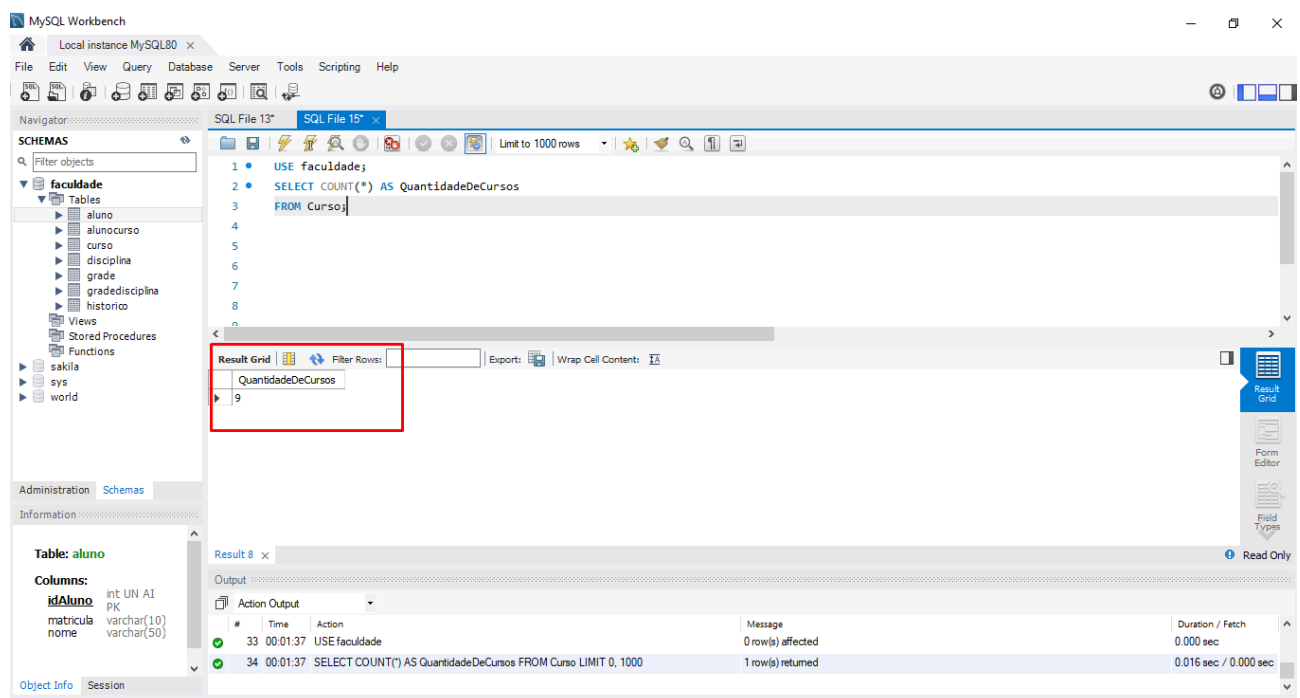
CREATE TABLE Historico (

```
id int unsigned NOT NULL auto_increment,  
idAluno int unsigned NOT NULL,  
idDisciplina int unsigned NOT NULL,  
nota float NOT NULL,  
dataHistorico date NOT NULL,  
PRIMARY KEY (id),  
constraint fk_historico_aluno foreign key (idAluno) references aluno(idAluno),  
constraint fk_historico_disciplina foreign key (idDisciplina) references  
disciplina(idDisciplina)  
);
```

Pontuação: 10 pontos.

2. Implemente uma consulta para listar o quantitativo de cursos existentes.

```
USE faculdade;  
SELECT COUNT(*) AS QuantidadeDeCursos  
FROM Curso;
```



The screenshot shows the MySQL Workbench interface. On the left, the 'Schemas' pane shows a tree view with 'faculdade' expanded, containing tables like 'aluno', 'alunocurso', 'curso', 'disciplina', 'grade', 'gradedisciplina', and 'historico'. The main editor window displays the SQL query: `USE faculdade;`, `SELECT COUNT(*) AS QuantidadeDeCursos`, and `FROM Curso;`. Below the query, the 'Result Grid' shows a single row with the value '9' under the column 'QuantidadeDeCursos'. The bottom pane shows the 'Output' tab with a table of results, including the query execution time and the number of rows affected and returned.

Table:	Columns:
aluno	idAluno int UN AI PK, matricula varchar(10), nome varchar(50)

Result Grid
QuantidadeDeCursos
9

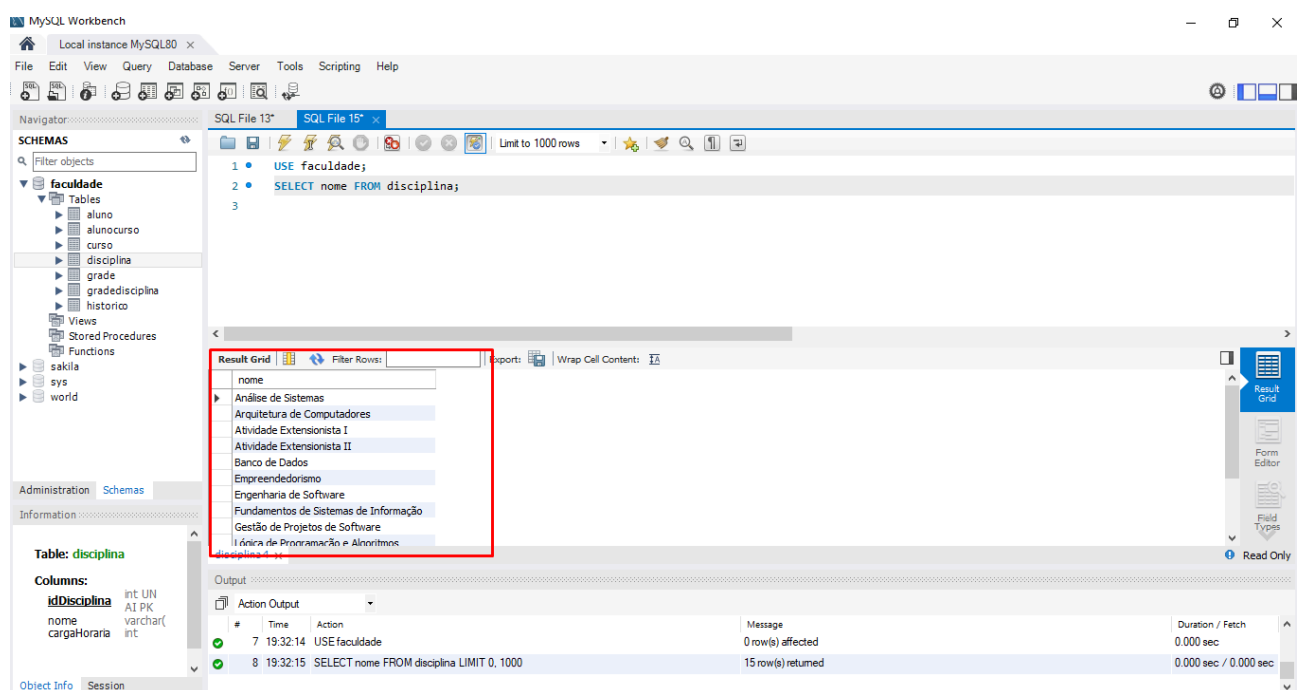
Output		
33 00:01:37 USE faculdade	0 row(s) affected	0.000 sec
34 00:01:37 SELECT COUNT(*) AS QuantidadeDeCursos FROM Curso LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec

Pontuação: 10 pontos.

3. Implemente uma consulta para listar o nome das disciplinas existentes.

USE faculdade;

SELECT nome FROM disciplina;



Pontuação: 10 pontos.

4. Implemente uma consulta para listar o nome de todos os cursos e o nome de seus respectivos alunos. A listagem deve ser mostrada em ordem decrescente pelo nome dos cursos.

USE faculdade;

**SELECT Curso.nome AS NomeDoCurso, Aluno.nome AS NomeDoAluno
FROM Curso**

JOIN AlunoCurso ON Curso.idCurso = AlunoCurso.idCurso

JOIN Aluno ON AlunoCurso.idAluno = Aluno.idAluno

ORDER BY Curso.nome DESC;

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
1 USE faculdade;
2 SELECT Curso.nome AS NomeDoCurso, Aluno.nome AS NomeDoAluno
3 FROM Curso
4 JOIN AlunoCurso ON Curso.idCurso = AlunoCurso.idCurso
5 JOIN Aluno ON AlunoCurso.idAluno = Aluno.idAluno
6 ORDER BY Curso.nome DESC;
```

The Results window displays the following data:

NomeDoCurso	NomeDoAluno
Redes de Computadores	Nicole Amanda de Jesus
Redes de Computadores	Vitor Martins
Jogos Digitais	Beatriz Leopoldina
Jogos Digitais	João Augusto de Moura
Gestão da Tecnologia da Informação	Miriam Miranda
Gestão da Tecnologia da Informação	Matheus Murilo de Souza
Engenharia de Software	Paulo Roberto Vitorino
Engenharia de Software	Mario Vicente
Engenharia da Computação	Viviane Chaves Filha
Engenharia da Computação	Antônio Crizer

The Output window shows the execution details:

#	Time	Action	Message	Duration / Fetch
27	23:44:23	USE faculdade	0 row(s) affected	0.000 sec
28	23:44:23	SELECT Curso.nome AS NomeDoCurso, Aluno.nome AS NomeDoAluno FROM Curso JOIN ...	19 row(s) returned	0.016 sec / 0.000 sec

Pontuação: 10 pontos.

5. Implemente uma consulta para listar o nome das disciplinas e a média das notas das disciplinas em todos os cursos. Para isso, utilize o comando *group by*.

USE faculdade;

SELECT Disciplina.nome AS NomeDaDisciplina, AVG(Historico.nota) AS MediaDasNotas

FROM Disciplina

JOIN Historico ON Disciplina.idDisciplina = Historico.idDisciplina

GROUP BY Disciplina.nome;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
1 USE faculdade;
2 SELECT Disciplina.nome AS NomeDaDisciplina, AVG(Historico.nota) AS MediaDasNotas
3 FROM Disciplina
4 JOIN Historico ON Disciplina.idDisciplina = Historico.idDisciplina
5 GROUP BY Disciplina.nome;
```

The Results window displays the following data:

NomeDaDisciplina	MediaDasNotas
Análise de Sistemas	85
Atividade Extensionista I	75
Banco de Dados	85
Gestão de Projetos de Software	75
Matemática Computacional	70
Programação de Computadores	70
Programação Orientada a Objetos	70
Sistema Gerenciador de Banco de Dados	82
Arquitetura de Computadores	76
Atividade Extensionista II	80

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
29	23:46:45	USE faculdade	0 row(s) affected	0.000 sec
30	23:46:45	SELECT Disciplina.nome AS NomeDaDisciplina, AVG(Historico.nota) AS MediaDasNotas FR...	11 row(s) returned	0.063 sec / 0.000 sec

Pontuação: 10 pontos.

6. Implemente uma consulta para listar o nome de todos os cursos e a quantidade de alunos em cada curso. Para isso, utilize os comandos *join* e *group by*.

USE faculdade;

SELECT Curso.nome AS NomeDoCurso, COUNT(AlunoCurso.idAluno) AS QuantidadeDeAlunos

FROM Curso

JOIN AlunoCurso ON Curso.idCurso = AlunoCurso.idCurso

GROUP BY Curso.nome;

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

SQL File 13* SQL File 15*

Limit to 1000 rows

SCHEMAS

Filter objects

faculdade

Tables

- aluno
- alunocurso
- curso
- disciplina
- grade
- gradedisciplina
- historico
- Views
- Stored Procedures
- Functions
- sakila
- sys
- world

Administration Schemas

Information

Table: aluno

Columns:

- idAluno int UN AI PK
- matricula varchar(10)
- nome varchar(50)

Object Info Session

```
1 USE faculdade;
2 SELECT Curso.nome AS NomeDoCurso, COUNT(AlunoCurso.idAluno) AS QuantidadeDeAlunos
3 FROM Curso
4 JOIN AlunoCurso ON Curso.idCurso = AlunoCurso.idCurso
5 GROUP BY Curso.nome;
```

Result Grid

NomeDoCurso	QuantidadeDeAlunos
Análise e Desenvolvimento de Sistemas	3
Banco de Dados	2
Ciência de Dados	2
Desenvolvimento Mobile	2
Engenharia da Computação	2
Engenharia de Software	2
Gestão da Tecnologia da Informação	2
Jogos Digitais	2
Redes de Computadores	2

Result 7 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
31	23:48:47	USE faculdade	0 row(s) affected	0.000 sec
32	23:48:47	SELECT Curso.nome AS NomeDoCurso, COUNT(AlunoCurso.idAluno) AS QuantidadeDeAlunos	9 row(s) returned	0.000 sec / 0.000 sec